

Bayesian Networks and Autoregressive Language Models — Completed Solution

Dataset and model The six sentences specified in the laboratory are used exactly as given. Tokens are lower-cased and each sentence receives `` and `` markers.

Question 1 The chain rule decomposes a sequence probability into conditional next-token probabilities. This makes generation practical: choose the first token, then repeatedly estimate or sample the next token conditioned on the preceding context.

Question 2 For the first-order network $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_T$, the assumption is: $P(X_t | X_1, \dots, X_{(t-1)}) = P(X_t | X_{(t-1)})$. Earlier tokens are conditionally irrelevant once the immediately previous token is known.

Question 3 — First-order CPT From the six training sentences: - $P(\text{the} | \text{START}) = 1$ - $P(\text{cat} | \text{the}) = 3/5 = 0.600$ - $P(\text{dog} | \text{the}) = 2/5 = 0.400$ - $P(\text{sat} | \text{cat}) = 2/3 = 0.667$ - $P(\text{ran} | \text{cat}) = 1/3 = 0.333$ - $P(\text{sat} | \text{dog}) = 2/3 = 0.667$ - $P(\text{ran} | \text{dog}) = 1/3 = 0.333$ - $P(\text{on} | \text{sat}) = 1$ - $P(\text{to} | \text{ran}) = 1$ - $P(\text{the} | \text{on}) = 1$ - $P(\text{the} | \text{to}) = 1$ - $P(\text{END} | \text{mat}) = P(\text{END} | \text{rug}) = P(\text{END} | \text{park}) = 1$. All unobserved transitions have probability zero.

Question 4 Transition counts are stored in a nested `defaultdict(Counter)`: current token -> counts of observed next tokens.

Question 5 For each current token, each count is divided by the total outgoing count: $P(\text{next} | \text{current}) = \text{count}(\text{current}, \text{next}) / \text{sum}_v \text{count}(\text{current}, v)$.

Question 6 Both required modes are implemented. Greedy generation chooses the maximum-probability token. Sampling draws randomly using the conditional probabilities as weights.

Question 7 An unseen context has no conditional distribution. The implementation returns `None` and safely stops generation rather than inventing a transition.

Question 8 A normalisation total of 0.87 means the conditional distribution is incorrect: probability mass is missing, or counting/normalisation is wrong. Every observed conditional distribution should sum to approximately 1.

Question 9 No. The most probable word is determined only by the training corpus and model assumptions. Human linguistic expectations use broader knowledge than this tiny corpus.

Question 10 Greedy generation follows the highest-probability path and therefore shows little variation. Sampling uses the whole conditional distribution and produces more variation.

Question 11 — Second-order model First-order uses $P(X_t | X_{(t-1)})$ and has a chain structure. Second-order uses $P(X_t | X_{(t-2)}, X_{(t-1)})$ with graph $X_{(t-2)} \rightarrow X_t \leftarrow X_{(t-1)}$. The second-order CPT is indexed by pairs of preceding tokens. It uses more context but needs more data because the number of possible contexts grows.

Question 12 More context can improve prediction because it distinguishes situations that a shorter context merges. At the same time, each context pair needs observations, so a small dataset produces sparse CPTs and many unseen/zero-probability contexts.

Question 13 A behavioural specification is preferable because it fixes the intended probabilistic model before implementation. It makes the representation, counting rule, conditional probabilities, generation procedure, stopping condition, and tests explicit. The generated code can then be checked against those requirements.

Prompt used "Implement a simple Python first-order autoregressive language model for the six-sentence lab dataset. Lower-case the sentences, add and , count consecutive-token transitions, construct $P(\text{next}|\text{current})$, display selected distributions, provide greedy prediction and probability-weighted sampling, stop at , and test that every conditional distribution sums to 1. Use only Python standard-library data structures and random sampling. Then implement a second-order model using $P(\text{next}|\text{previous_previous}, \text{previous})$ and counts of observed triples. Do not use an ML library or pretrained model."

Verification I verified transition counts against the supplied sentences, checked that each conditional distribution normalises to 1, checked that greedy and sampling modes behave differently, and checked that the second-order implementation actually uses two-token contexts.

Question 14 Viewing the model as a Bayesian network provides: (1) an explicit representation of dependencies, (2) a factorisation of the joint distribution, (3) an interpretation of each CPT row as a conditional probability distribution, (4) a principled sampling procedure, (5) explicit independence assumptions, (6) a clear way to see the effect of increasing context, and (7) concrete probability invariants for testing.

Results All saved execution outputs show conditional-probability normalisation totals of 1.0 (up to floating-point precision). The files also contain five greedy generations and twenty fixed-seed sampling generations for each model.

The first-order model has 10 observed context rows and 13 observed transition entries. The second-order model has 17 observed context-pair rows and 18 observed transition entries. The second-order model therefore carries more contextual information but is sparser on this small corpus.

Final reflection The LLM is useful as an engineering assistant when the probabilistic behaviour is specified first. Human verification remains necessary because runnable code can still implement the wrong model. In this lab, the key checks were transition counts, conditional-probability normalisation, generation behaviour, and confirmation that the second-order implementation really conditions on two previous tokens.